

Deep Learning Lab Manual CAI2503, Prepared By IC: Pakruddin.B, Asst.Prof., PSCSE,PU. 2025-26

✓ Lab 7: Regularization in Neural Networks

Objective: Compare L1, L2, and Dropout regularization in MLP on MNIST

Duration: ~2 hours

What Students Will Learn

How L1 regularization encourages sparsity in weights.

How L2 regularization penalizes large weights to improve generalization.

How Dropout prevents co-adaptation of neurons.

How different techniques affect training vs. validation performance.

Expected Outputs

Validation accuracy plot comparing No Regularization, L1, L2, Dropout.

Validation loss plot showing overfitting control.

Printed table showing final test accuracies for each method.

```
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
import matplotlib.pyplot as plt

# 1. Load and preprocess MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten images
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

# 2. Function to build MLP with specific regularization
def build_mlp(reg_type=None, reg_value=0.001, dropout_rate=0.0):
    if reg_type == 'l1':
        kernel_reg = regularizers.l1(reg_value)
    elif reg_type == 'l2':
        kernel_reg = regularizers.l2(reg_value)
    else:
        kernel_reg = None

    model = models.Sequential([
        layers.Input(shape=(784,)),
        layers.Dense(128, activation='relu', kernel_regularizer=kernel_reg),
        layers.Dropout(dropout_rate),
        layers.Dense(64, activation='relu', kernel_regularizer=kernel_reg),
        layers.Dropout(dropout_rate),
        layers.Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    return model

# 3. Configurations to test
configs = [
    {'reg_type': None, 'reg_value': 0.0, 'dropout_rate': 0.0},
    {'reg_type': 'l1', 'reg_value': 0.001, 'dropout_rate': 0.0},
    {'reg_type': 'l2', 'reg_value': 0.001, 'dropout_rate': 0.0},
    {'reg_type': None, 'reg_value': 0.0, 'dropout_rate': 0.3},
    {'reg_type': 'l2', 'reg_value': 0.001, 'dropout_rate': 0.3}
]

history_results = {}
test_results = {}

# 4. Train and evaluate each configuration
```

```

# 4. Train and evaluate each configuration
for cfg in configs:
    print(f"\nTraining with {cfg}")
    model = build_mlp(cfg['reg_type'], cfg['reg_value'], cfg['dropout_rate'])
    history = model.fit(x_train, y_train,
                        validation_split=0.2,
                        epochs=10,
                        batch_size=64,
                        verbose=0)
    history_results[str(cfg)] = history.history
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    test_results[str(cfg)] = test_acc
    print(f"Test Accuracy: {test_acc:.4f}")

# 5. Plot validation accuracy curves
plt.figure(figsize=(8, 6))
for cfg, hist in history_results.items():
    plt.plot(hist['val_accuracy'], label=cfg)
plt.title("Validation Accuracy - Regularization Techniques")
plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# 6. Plot validation loss curves
plt.figure(figsize=(8, 6))
for cfg, hist in history_results.items():
    plt.plot(hist['val_loss'], label=cfg)
plt.title("Validation Loss - Regularization Techniques")
plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid(True)
plt.show()

# 7. Final test accuracy comparison
print("\nFinal Test Accuracy Comparison:")
for cfg, acc in test_results.items():
    print(f"{cfg}: {acc:.4f}")

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 1s 0us/step

Training with {'reg_type': None, 'reg_value': 0.0, 'dropout_rate': 0.0}
Test Accuracy: 0.9763

Training with {'reg_type': 'l1', 'reg_value': 0.001, 'dropout_rate': 0.0}
Test Accuracy: 0.9494

Training with {'reg_type': 'l2', 'reg_value': 0.001, 'dropout_rate': 0.0}
Test Accuracy: 0.9699

Training with {'reg_type': None, 'reg_value': 0.0, 'dropout_rate': 0.3}
Test Accuracy: 0.9792

Training with {'reg_type': 'l2', 'reg_value': 0.001, 'dropout_rate': 0.3}
Test Accuracy: 0.9698

